

实验总结

本实验主要采用 API 调用接口的方式实现音乐数据的爬取，并使用 Django 框架搭建了一个简单的 Web 网页，对爬取到的歌手、歌曲及歌词等数据进行展示。在数据分析部分，使用 pandas 对数据进行整理和统计，并结合 matplotlib 等 Python 库进行数据可视化，从歌曲发布时间、歌曲时长以及歌词文本等不同角度对数据进行了进一步分析。

1. 爬虫的实现

本实验最终收集了 **166 名歌手和约 3520 首歌曲** 的数据。最初设置的爬取范围为 400 名歌手，在实际爬取过程中，由于部分歌手无法正常获取数据，或者缺少有效的歌手简介等信息，因此对数据进行了筛选，最终保留了 166 名符合要求的歌手。在爬虫实现方面，本实验主要使用 `requests` 库完成网络请求，并通过音乐平台提供的 API 接口获取歌手、歌曲、歌词等相关数据。与传统的通过 HTML 页面进行解析的方式不同，本实验没有将 HTML 解析作为主要的数据获取方式，而是直接对 API 返回的数据进行处理。在项目初期，原计划使用 QQ 音乐作为数据来源，但实际爬取过程中遇到了较为严格的反爬机制，不管是 HTML 还是 API 接口均尝试无果。在多次尝试后，考虑到数据获取的稳定性以及课程作业的实际需求，最终将数据源更换为网易云音乐，并采用其相关 API 接口完成数据获取。在 API 接口的寻找和使用过程中，参考了开源项目 [NeteaseCloudMusicApi](#) 中提供的接口信息。实践过程中发现，其中部分已经标记为废弃的接口在实际环境中仍然可以正常使用，因此选择其中能够满足本实验需求的接口进行数据获取。需要说明的是，本实验主要参考了该项目的 **接口信息和调用方式**，并未直接使用其爬虫程序。数据请求、数据解析、异常处理、数据保存以及后续的数据处理代码均由本人根据实验需求独立完成。

2. Web 网页的实现

在数据爬取完成后，本实验使用 Django 框架搭建了一个简单的音乐数据展示网站，将爬取到的数据导入 SQLite 数据库，并通过 Django 的 Model、View 和 Template 等组件实现数据展示。网页主要实现了歌手列表、歌手详情、歌曲列表、歌曲详情以及搜索等功能，使爬取到的原始数据能够通过较为直观的网页界面进行浏览。在搜索功能中，根据歌曲名称、歌手名称等信息计算匹配程度，并采用 **权重排序** 的方式对搜索结果进行排序，使匹配程度较高的结果优先显示。考虑到本实验的数据规模相对较小，没有额外引入较为复杂的搜索或排序算法，而是根据实际需求设计了较为简单的权重计算方式。在测试范围内，搜索及排序操作的响应时间均小于 **1秒**，能够满足本实验网站的基本使用需求。

3. 数据分析

在完成数据爬取和网站搭建后，本实验进一步对收集到的数据进行了统计分析。首先，从歌曲发布时间的角度分析了不同年代歌曲的分布情况，发现样本中的歌曲主要集中在 2000 年以后，其中 2010 年代的歌曲数量最多，体现出较明显的年代集中趋势。其次，根据歌词信息识别纯音乐歌曲，并将其与普通歌曲进行对比。分析发现，纯音乐与普通歌曲在平均时长、中位数以及时长分布等方面均存在一定差异，其中纯音乐整体具有更长的时长，并且长时长歌曲的比例明显更高。最后，对歌曲歌词进行分词和清洗，并分别统计词语的总出现次数以及包含该词的歌曲数量。通过 Pearson 相关系数对两个指标进行分析，得到相关系数约为 **0.9557**，表明在本次样本中歌词词频与歌曲覆盖范围之间存在较强的正相关关系。通过以上分析，不仅对爬取到的数据进行了简单的统计，也尝试从数据中进一步发现不同变量之间的关系和规律。

4. 实验过程中遇到的问题与解决方法

本实验过程中遇到的主要问题是**数据源的选择以及反爬机制**。在最初尝试使用QQ音乐进行数据爬取时，发现其存在较为严格的访问限制。经过多次尝试后，如果继续使用原有方案，需要投入较多精力处理反爬相关问题，同时也会增加实验实现的复杂程度。因此最终选择更换数据源，将爬取对象调整为网易云音乐，并通过其API接口获取数据。这一过程也使我认识到，实际进行网络数据采集时，数据源的选择与爬虫代码本身同样重要。对于课程实验而言，在满足数据需求的前提下，选择稳定、结构清晰的数据接口，可以将更多精力放在数据处理、分析以及程序设计本身。另外，在数据整理过程中也发现，原始数据中存在部分缺失、格式不统一以及无效数据等情况，因此在数据保存和分析之前需要进行筛选和清洗。这一过程也保证了后续统计分析结果的可靠性。

5. 实验总结

通过本次实验，我完成了从**数据获取、数据处理、数据存储、Web展示到数据分析**的一套完整流程。在爬虫部分，掌握了使用 `requests` 调用API获取数据，并根据实际情况处理数据获取过程中的异常问题；在Web部分，进一步熟悉了Django的基本开发流程，实现了歌曲和歌手数据的展示以及搜索功能；在数据分析部分，则学习了使用pandas对数据进行整理，并利用matplotlib对统计结果进行可视化。相比单独实现一个爬虫程序，本实验让我对数据从产生到使用的完整流程有了更加直观的认识。尤其是在实际爬取过程中遇到反爬机制、数据缺失以及数据清洗等问题后，也进一步认识到，网络爬虫并不仅仅是发送请求和获取网页内容，还需要根据实际数据源不断调整方案。最终，本实验基本完成了预期的功能，并通过数据分析进一步挖掘了爬取数据中的一些规律，使整个项目不仅能够完成数据采集，也能够实现对数据的进一步利用。